

---

# CANDIDATE ASSESSMENT

## Multi-Agent System for Customer Support

---

|                   |                                                                             |
|-------------------|-----------------------------------------------------------------------------|
| Role              | AI Engineering Intern                                                       |
| Assessment Type   | Take-Home                                                                   |
| Total Duration    | Take-Home: up to 5 days                                                     |
| Submission Format | GitHub repository (private, invite shared) or ZIP archive and Live Demo URL |
| Language / Stack  | Python (preferred) — any LLM framework permitted                            |

Please read this entire document carefully before starting. The assessment tests system design thinking and working code , not memorized answers. You are encouraged to use any documentation, references, or AI coding tools during the take-home portion.

---

# 1. Problem Statement

---

You are tasked with building a prototype multi-agent customer support system that can handle end-to-end customer interactions for a fictional SaaS product called "CloudDash" — a cloud infrastructure monitoring platform.

The system must:

- Accept a customer query via a simple API endpoint or CLI interface.
- Route the query to the appropriate specialized agent based on intent classification.
- Retrieve relevant information from a knowledge base to ground agent responses.
- Seamlessly hand over context between agents when a conversation crosses domain boundaries.
- Escalate to a human operator (simulated) when the system cannot resolve the issue.

## 1.1 Product Context — CloudDash

CloudDash is a B2B SaaS platform that provides real-time monitoring, alerting, and cost optimization for cloud infrastructure (AWS, GCP, Azure). Customers contact support for issues including:

- Technical issues — alert configuration, integration setup, dashboard errors, API usage.
- Billing & subscription — plan upgrades/downgrades, invoice disputes, payment failures.
- Account management — user access, SSO setup, team/org management.
- General inquiries — feature requests, product roadmap questions, onboarding help.

---

# 2. Functional Requirements

---

## 2.1 Agent Architecture

Design and implement the following agents. You may add more if your design calls for it, but these are the minimum:

| Agent                   | Responsibility                                                                            | Key Capabilities                                                    |
|-------------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Triage Agent            | First point of contact. Classifies intent, gathers initial context, routes to specialist. | Intent classification, entity extraction, routing logic             |
| Technical Support Agent | Resolves technical issues using KB articles, troubleshooting guides, and API docs.        | KB retrieval, step-by-step troubleshooting, code snippet generation |
| Billing Agent           | Handles billing inquiries, explains invoices, processes plan changes (simulated).         | Account lookup (mock), plan comparison, policy citation             |
| Escalation Agent        | Packages conversation context and routes to human support when AI cannot resolve.         | Context summarization, priority classification, handover packaging  |

---

## 2.2 Knowledge Base Integration

Build a RAG (Retrieval-Augmented Generation) pipeline that ingests a knowledge base and grounds agent responses in retrieved content. Specifically:

- Ingest the provided sample knowledge base documents (FAQ, troubleshooting guides, billing policies — see Section 5 for sample data structure).
- Implement chunking, embedding, and indexing into a vector store of your choice (FAISS, ChromaDB, Pinecone, Qdrant, etc.).
- Implement a retrieval chain that rewrites the user query using conversation context before searching.
- Retrieved chunks must be cited or referenced in the agent's response so the customer can verify the source.

**Bonus:** Implement hybrid retrieval (vector + keyword/BM25) and a re-ranking step. This is not required but will be positively noted.

## 2.3 Agent Handover Protocol

When a conversation crosses domain boundaries (e.g., a technical issue leads to a billing question), the system must hand over to the appropriate agent. Your handover mechanism must:

- Preserve full conversation history or a structured summary for the receiving agent.
- Transfer extracted entities (customer ID, issue type, product references) without loss.
- Allow the receiving agent to acknowledge the handover and continue seamlessly — the customer should not need to repeat information.
- Handle failed handovers gracefully — if the target agent rejects or errors, the system should fall back to the Triage Agent or Escalation Agent.
- Log every handover event with: timestamp, source agent, target agent, reason, and context snapshot.

## 2.4 Conversation Interface

Provide one of the following interfaces for interacting with the system:

- **Option A (Recommended):** A simple REST API with endpoints for starting a conversation, sending messages, and retrieving conversation history.
- **Option B:** A CLI-based chat interface that runs in the terminal.
- **Option C (Bonus):** A minimal web UI (Streamlit, Gradio, or basic React) for visual demonstration.

---

## 3. Non-Functional Requirements

---

### 3.1 Code Quality & Structure

- Clean separation of concerns — agent logic, routing, KB retrieval, and API/interface should be in separate modules.
- Typed data models (Pydantic or dataclasses) for conversation state, handover payloads, and agent responses.
- Meaningful error handling — not bare try/except blocks. Errors should be logged with context.
- A README.md with: setup instructions, architecture overview, design decisions, and known limitations.

### 3.2 Observability & Logging

- Structured logging (JSON preferred) for every agent invocation, KB retrieval, and handover event.
- Each conversation should have a unique trace ID that connects all logs for that session.

Bonus: Integrate an LLM observability tool (Langfuse, LangSmith, Arize Phoenix, or similar) to trace agent calls, token usage, and latency.

### 3.3 Guardrails & Safety

- Implement at least one input guardrail (e.g., prompt injection detection or off-topic filtering).
- Implement at least one output guardrail (e.g., PII redaction, hallucination check against retrieved KB content, or policy-violation filter).
- The system should never fabricate information about CloudDash pricing, policies, or features — if the KB does not contain the answer, the agent should acknowledge this and offer to escalate.

### 3.4 Configuration & Extensibility

- Agent definitions (system prompts, available tools, routing rules) should be configurable via YAML/JSON — not hardcoded in application logic.
- Adding a new agent type should not require modifying the core orchestration code.

---

## 4. Evaluation Criteria

---

Your submission will be evaluated on the following dimensions:

| Criteria                     | Weight | What We Look For                                                                   |
|------------------------------|--------|------------------------------------------------------------------------------------|
| System Design & Architecture | 25%    | Clean decomposition, orchestration pattern choice, state management, extensibility |
| Knowledge Base Integration   | 25%    | RAG pipeline quality, chunking strategy, retrieval accuracy, citation in responses |

---

|                                              |            |                                                                                    |
|----------------------------------------------|------------|------------------------------------------------------------------------------------|
| <b>Agent Handover Implementation</b>         | <b>20%</b> | Context preservation, seamless transitions, failure handling, audit logging        |
| <b>Code Quality &amp; Engineering</b>        | <b>15%</b> | Clean code, typed models, error handling, tests, documentation                     |
| <b>Guardrails &amp; Production Readiness</b> | <b>15%</b> | Input/output safety, graceful degradation, observability, configuration management |

Important: We value working code over perfect code. A functional prototype with clear design decisions and documented trade-offs will score higher than an over-engineered solution that doesn't run.

---

## 5. Sample Knowledge Base Structure

---

You are expected to create your own sample knowledge base content for CloudDash. Below is the expected structure and a few example entries to guide you. Create at least 15–20 articles across these categories:

### 5.1 Expected Document Categories

| Category                   | Description & Examples                                                                                                          |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| FAQs                       | Common questions: "How do I reset my API key?", "What cloud providers does CloudDash support?", "How do I invite team members?" |
| Troubleshooting Guides     | Step-by-step resolution: "Alerts not firing", "Dashboard loading slowly", "Integration with AWS CloudWatch failing"             |
| Billing & Pricing Policies | Plan details, upgrade/downgrade rules, refund policy, invoice explanation, payment methods                                      |
| API Documentation          | Endpoint references, authentication, rate limits, webhook configuration, SDK usage                                              |
| Account & Access           | SSO setup, RBAC configuration, team management, audit logs                                                                      |

### 5.2 Sample Article Format

Each KB article should follow this structure (JSON, Markdown, or YAML — your choice):

```
{
  "id": "KB-001",
  "title": "How to Configure Alert Thresholds",
  "category": "troubleshooting",
  "tags": ["alerts", "configuration", "thresholds"],
  "content": "Step-by-step guide content here...",
  "last_updated": "2026-04-15",
  "applies_to": ["Pro", "Enterprise"]
}
```

---

## 6. Test Scenarios

---

Your system should be able to handle the following scenarios. We will use these (and similar variations) during the live discussion to evaluate your implementation:

### Scenario 1 — Single-Agent Resolution

Customer: "My CloudDash alerts stopped firing after I updated my AWS integration credentials yesterday. I'm on the Pro plan."

Expected: Triage classifies as technical issue → routes to Technical Support Agent → agent retrieves relevant KB article on AWS integration + alert configuration → provides step-by-step resolution with source citation.

---

## Scenario 2 — Cross-Agent Handover

Customer: "I want to upgrade from Pro to Enterprise, but first can you check if the SSO integration issue I reported last week has been resolved?"

Expected: Triage identifies two intents → routes to Technical Support for SSO issue → after resolution, hands over to Billing Agent for upgrade → Billing Agent has full context of prior discussion.

## Scenario 3 — Escalation to Human

Customer: "I've been charged twice for April. I need an immediate refund and I want to speak to a manager."

Expected: Triage routes to Billing Agent → Billing Agent attempts resolution but recognizes refund authority is limited → escalates to Escalation Agent → Escalation Agent packages context (customer ID, billing issue, urgency: high, sentiment: frustrated) and triggers human handover.

## Scenario 4 — KB Retrieval Failure

Customer: "Does CloudDash support integration with Datadog for cross-platform alerting?"

Expected: Agent searches KB → finds no relevant article → transparently acknowledges the limitation → offers to escalate to the product team or create a feature request.

---

## 7. Submission Guidelines

---

### 7.1 Repository Structure (Recommended)

| File / Directory | Purpose                                                   |
|------------------|-----------------------------------------------------------|
| README.md        | Setup, architecture diagram, design decisions, trade-offs |
| agents/          | Agent implementations — one file per agent + orchestrator |
| knowledge_base/  | Sample KB documents + ingestion script                    |
| retrieval/       | RAG pipeline — embedding, indexing, retrieval, re-ranking |
| handover/        | Handover protocol, context packaging, audit logging       |
| api/ or cli/     | Interface layer — REST API or CLI chat                    |
| config/          | Agent configs, prompts, routing rules (YAML/JSON)         |
| tests/           | Unit and integration tests                                |
| requirements.txt | Python dependencies                                       |

### 7.2 What to Submit

- A working codebase that can be set up and run with the instructions in your README.
- Environment variables for API keys should be documented (use .env.example).
- Your sample knowledge base (15–20 articles minimum).
- A brief architecture document or diagram (can be in README or a separate [ARCHITECTURE.md](#)).
- Production-grade infrastructure is not required; however, candidates must provide a working public deployment accessible via a live URL. Simple free-tier hosting solutions are completely acceptable.

### 7.3 What NOT to Worry About

- Production-grade deployment (Docker, CI/CD, Kubernetes)
- A polished UI — we are evaluating backend architecture and agent logic, not frontend design.
- Perfect prompt engineering — reasonable prompts that demonstrate intent are sufficient.
- 100% test coverage — a few meaningful tests demonstrating your testing approach are enough.

---

## 8. Live Discussion (45 minutes)

---

After reviewing your submission, we will schedule a live technical discussion. Be prepared to:

- Walk through your architecture decisions and explain the trade-offs you made.



- 
- Run your system live and demonstrate the test scenarios from Section 6.
  - Discuss how you would evolve this system for production scale (multi-tenancy, rate limiting, cost optimization, monitoring).
  - Answer follow-up questions on failure modes, edge cases, and alternative approaches.
  - Discuss how you would add a new agent type (e.g., an Onboarding Agent) without modifying existing code.

If you have questions about the assessment requirements, please reach out before starting. We value clarity and would rather answer questions upfront than have you make incorrect assumptions.

---

*Good luck! We look forward to seeing your design.*